Spec-Driven Development

General Agents, Chapter 6/12 — Revision & Viva-Prep Cheatsheet (English + Roman Urdu)
Source: agentfactory.panaversity.org/docs/spec-driven-development

EN: 'Agree on the what before you generate the how.' A thinking discipline, not a coding skill. Anthropic data: across ~400K sessions, people do 70% of planning themselves and 80% of execution goes to the agent -- and domain expertise, not coding skill, predicts success best.
Roman Urdu: 'How generate karne se pehle what pe agree karo.' Coding skill nahi, thinking discipline hai. Anthropic data: ~400K sessions mein log planning ka 70% khud karte, execution ka 80% agent ko -- aur domain expertise, coding skill nahi, success ka sabse bara predictor hai.

1. Vibe Coding vs SDD + Spec as Product

- The difference is WHEN you think: vibe coding thinks while building (cost arrives later, all at once); SDD thinks first, writes it down, then builds (mostly mechanical).
- Old: spec -> build -> throw away. SDD: spec is durable, code regenerates from it. Test for a vague spec line: 'could a competent person satisfy this line while building the wrong thing?'
- **3 levels:** Spec-First (default), Spec-Anchored (maintained for months), Spec-as-Source (mature teams -- biggest claim, least evidence).

2. The Method: Constitution + 4 Phases

- **Constitution** (CLAUDE.md/AGENTS.md) = persistent context, not enforced law -- back it with tests/hooks/CI. Test: 'would removing this line let AI make a mistake?'
- **Phase 1 Research** -> **Phase 2 Specify** (Goal, scenarios, requirements, edge cases, Out of scope -- never skip, acceptance criteria) -> **Phase 3 Clarify** (have AI interview YOU -- highest value, most skipped) -> **Phase 4 Build** (process matches change size).

3. Judgment: When to Use It

- Use SDD for: multi-file, long-maintained, expensive-if-wrong, fuzzy requirements, multiple stakeholders. Skip/vibe for: one-off scripts, cheap undos, clear tasks.
- **Keep the spec alive:** it's only source of truth while it stays true -- any code change must update spec.md in the SAME commit (Spec-First -> Spec-Anchored).
- **Valentina Servile's 3 counter-points** against 'spec is the only language': natural language gets ambiguous, agents are non-deterministic, agent-written code decays.

Key Terms Cheatsheet

One line each — English + Roman Urdu.

Term	English	Roman Urdu
Spec-Driven Development (SDD)	Agree on the what before generating the how.	How generate karne se pehle what pe agree karo.
Constitution	The persistent-context rules file for all features (not enforced law).	Sab features ke liye persistent-context rules file (enforced law nahi).
Clarify phase	Having the AI interview YOU about ambiguities -- most valuable, most skipped step.	AI se khud ko ambiguities par interview karwana -- sabse valuable, sabse zyada skip hone wala step.
Out of scope section	A spec section that's never allowed to be skipped.	Spec section jo kabhi skip nahi hona chahiye.
Spec drift	When code changes but the spec doc doesn't -- the spec becomes a lie.	Jab code badle lekin spec doc na badle -- spec jhoothi ban jati hai.

Quick Self-Check

Question	Answer
What's the core difference between vibe coding and SDD?	When you think -- vibe coding thinks while building; SDD thinks first and writes it down before building.
Why is the Clarify phase called the most valuable and most skipped step?	Having the AI interview you surfaces unstated assumptions cheaply -- skipping it lets small ambiguities become real bugs later.
What predicted success best in Anthropic's ~400K session data?	Domain expertise -- not coding skill.
What's the fix for 'spec drift' (code changed, spec didn't)?	Any code change updates spec.md in the same commit -- moving from Spec-First toward Spec-Anchored discipline.

Where This Leads / Sources

Leads to: The Four Layers, Eval-Driven Development. Full 6-project practice set (capstone: Stranger Test) in 05-practice.

Source: docs/spec-driven-development/ (this repo), content originally via Zia Tutor AI connector.